

第四章 第一节 Python进阶篇——面向对象：类/对象

- **Python 类/对象**

Python 是一种面向对象的编程语言。

Python 中的几乎所有东西都是对象，拥有属性和方法。

类（Class）类似对象构造函数，或者是用于创建对象的“蓝图”。

- **创建类**

如需创建类，请使用 `class` 关键字：

实例

使用名为 `x` 的属性，创建一个名为 `MyClass` 的类：

```
class MyClass:
```

```
    x = 5
```

```
print(MyClass)
```

```
<class '__main__.MyClass'>
```

第四章 第一节 Python进阶篇——面向对象：类/对象

- **创建对象**

现在我们可以使用名为 myClass 的类来创建对象：

实例

创建一个名为 p1 的对象，并打印 x 的值：

```
p1 = MyClass()
```

```
print(p1.x)
```

5

第四章 第一节 Python进阶篇——面向对象：类/对象

- **__init__() 函数**

上面的例子是最简单形式的类和对象，在实际应用程序中并不真正有用。

要理解类的含义，我们必须先了解内置的 `__init__()` 函数。

所有类都有一个名为 `__init__()` 的函数，它始终在启动类时执行。

使用 `__init__()` 函数将值赋给对象属性，或者在创建对象时需要执行的其他操作：

实例

创建名为 `Person` 的类，使用 `__init__()` 函数为 `name` 和 `age` 赋值：

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
p1 = Person("Bill", 63)
```

```
print(p1.name)
```

```
print(p1.age)
```

```
Bill
```

```
63
```

注释：每次使用类创建新对象时，都会自动调用 `__init__()` 函数。

第四章 第一节 Python进阶篇——面向对象：类/对象

- **str() 函数**

str() 函数控制当类对象表示为字符串时应返回什么。如果未设置 str() 函数，则返回对象的字符串表示：

示例，没有 str() 函数的对象的字符串表示：

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
p1 = Person("John", 36)
print(p1)
```

→ /workspace cd /workspace && python 2m7vf.py

<__main__.Person object at 0x7fc8fa7c8f10>

→ /workspace

第四章 第一节 Python进阶篇——面向对象：类/对象

- **str() 函数**

示例，带有 str() 函数的对象的字符串表示：

```
class Person:
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
    def __str__(self):
```

```
        return f"{self.name}({self.age})"
```

```
p1 = Person("John", 36)
```

```
print(p1)
```

→ /workspace cd /workspace && python 8akrt.py

John(36)

→ /workspace

第四章 第一节 Python进阶篇——面向对象：类/对象

- 对象方法

对象也可以包含方法。对象中的方法是属于对象的函数。让我们在 **Person** 类中创建一个方法：示例，插入一个打印问候的函数，并在 **p1** 对象上执行它：

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def myfunc(self):
        print("Hello my name is " + self.name)
```

```
p1 = Person("John", 36)
p1.myfunc()
```

Hello my name is John

第四章 第一节 Python进阶篇——面向对象：类/对象

- **self 参数**

self 参数是对类的当前实例的引用，并用于访问属于类的变量。它不一定要命名为 **self**，您可以根据需要命名它，但它必须是类中任何函数的第一个参数：

示例，使用 **mysillyobject** 和 **abc** 代替 **self**：

```
class Person:  
    def __init__(mysillyobject, name, age):  
        mysillyobject.name = name  
        mysillyobject.age = age
```

```
def myfunc(abc):  
    print("Hello my name is " + abc.name)
```

```
p1 = Person("John", 36)  
p1.myfunc()
```

Hello my name is John

第四章 第一节 Python进阶篇——面向对象：类/对象

- 修改对象属性

您可以像这样修改对象的属性：

示例，将 **p1** 的年龄设置为 **40**：

```
p1.age = 40
```

- 删除对象属性

您可以使用 **del** 关键字来删除对象的属性：

示例，从 **p1** 对象中删除 **age** 属性：

```
del p1.age
```

- 删除对象

您可以使用 **del** 关键字来删除对象：

示例，删除 **p1** 对象：

```
del p1
```


第四章 第一节 Python进阶篇——面向对象：类/对象

- **pass 语句**

类定义不能空，但如果由于某种原因类定义没有内容，请插入 `pass` 语句以避免出错。

示例

```
class Person:  
    pass
```

- **类继承**

```
Class MyClass(BaseClass):
```

```
    #类定义
```

```
    pass
```

第四章 第一节 Python进阶篇——面向对象：类/对象

- **pass 语句**

类定义不能空，但如果由于某种原因类定义没有内容，请插入 `pass` 语句以避免出错。

示例

```
class Person:  
    pass
```

- **类继承**

```
Class MyClass(BaseClass):
```

```
    #类定义
```

```
    pass
```